Mentimeter

Laboratorul 4. Static și final. Singleton Design Pattern

Mentimeter

Cuvântul cheie final

- variabilele declarate cu final pot fi inițializate o singură dată
- daar obiectele către care pointează pot fi modificate intern

1

Mentimeter

```java
class Student {
    private final String name;

    public Student(String name) {
        this.name = name;
    }

    public void setName(String name) {
        this.name = name;
    }
}
```

Se generează eroare sau nu?

Mentimeter

Obiecte immutable

- toate atributele unui obiect admit o unică inițializare
- nu putem schimba obiectul în sine (ex: informația pe care o stochează), ci doar referința către un alt obiect
- exemple: instanțele claselor String și Integer

Mentimeter

Cum prelucrăm astfel de obiecte?

→ prin instanțierea de noi obiecte și nu prin alterarea obiectelor înseși

Mentimeter

```java
public static void main(String[] args) {
    String s1 = "POO";
    String s2 = s1.toLowerCase();
    System.out.println(s1 == s2);
}
```

Exemplu String 1

Mentimeter

```java
public static void main(String[] args) {
    String s1 = "POO";
    String s2 = s1.toUpperCase();
    System.out.println(s1 == s2);
    System.out.println(s1.equals(s2));
}
```

Exemplu String 2

Mentimeter

String pool

- obiecte de tip String sunt immutable
- literalii de tip String sunt păstrați într-un pool
- limitează memoria utilizată prin inițializarea unui singur obiect către care pot pointa mai multe referințe

Mentimeter

```java
public static void main(String[] args) {
    String s1 = "abc";
    String s2 = "abc";
    System.out.println(s1 == s2);
    System.out.println(s1.equals(s2));

    String s3 = new String( original: "abc");
    String s4 = new String( original: "abc");
    System.out.println(s3 == s4);
    System.out.println(s3.equals(s4));
}
```

Exemplu String pool

Mentimeter

```java
public static String concatenationUsingTheStringClass() {
    String t = "Java";
    for (int i = 0; i < 10000; i++) {
        t = t + "POO";
    }
    return t;
}
```

Probleme

Mentimeter

De ce e o problemă?

- nu ne pasă decât de rezultatul final
- umplem pool-ul cu string-uri intermediare pe care nu le mai folosim

Mentimeter

StringBuilder

- avem un singur StringBuilder și nu câte un String pentru fiecare pas intermediar
- reduce overheadul

Mentimeter

```java
public static String concatenationUsingTheStringBuilderClass() {
    StringBuilder sb = new StringBuilder("Java");
    for (int i = 0; i < 10000; i++) {
        sb.append("POO");
    }
    return sb.toString();
}
```

Soluție folosind StringBuilder

3

Mentimeter

Alte utilizări ale cuvântului final

- aplicat unei clase - NU poate fi extinsă
- aplicat unei metode - NU poate fi suprascrisă

Mentimeter

Cuvântul cheie static

- folosit pentru câmpuri/metode independente de instanțele clasei
- vor ocupa o locație unică în memorie, care nu depinde de obiectele create din clasa respectivă

1

Mentimeter

```java
public class Student {
    private String name;
    private static int UNIVERSITY_CODE = 120;
    private static int STUDENT_COUNT = 0;

    public Student(String name) {
        this.name = name;
        STUDENT_COUNT++;
    }

    public static int getStudentCount() {
        return STUDENT_COUNT;
    }
}
```

Exemplu câmp static

Mentimeter

```java
package com.company;


public class Main {
    public static void main(String[] args) {
        Student student1 = new Student( name: "Rick");
        System.out.println(Student.getStudentCount());


        Student student2 = new Student( name: "Morty");
        System.out.println(Student.getStudentCount());
    }
}
```

Ce se va afișa?

Mentimeter

Apelarea metodelor statice

- instance.staticMethod()
- ClassName.staticMethod()
- recomandată e a doua variantă pentru că e mult mai clar că metoda nu depinde de o instanță a clasei

2

Mentimeter

```java
public class Student {
    private String name;
    private static int UNIVERSITY_CODE;
    private static int STUDENT_COUNT;

    static {
        UNIVERSITY_CODE = 120;
        STUDENT_COUNT = 0;
    }

    public Student(String name) {
        this.name = name;
        STUDENT_COUNT++;
    }

    public static int getStudentCount() {
        return STUDENT_COUNT;
    }
}
```

Blocuri statice de cod

Mentimeter

Blocurile statice de cod

- apelate când clasa este încărcată
- încărcarea se face la prima referire în cod

Mentimeter

Singleton Pattern

restricționează numărul de
instanțieri la un singur obiect

2

Mentimeter

Utilizări

- în locul variabilelor globale
- obiecte de tip logger
- resurse partajate (ex: același fișier de input folosit de mai multe clase)
- în alte design pattern-uri (ex: Factory)

1

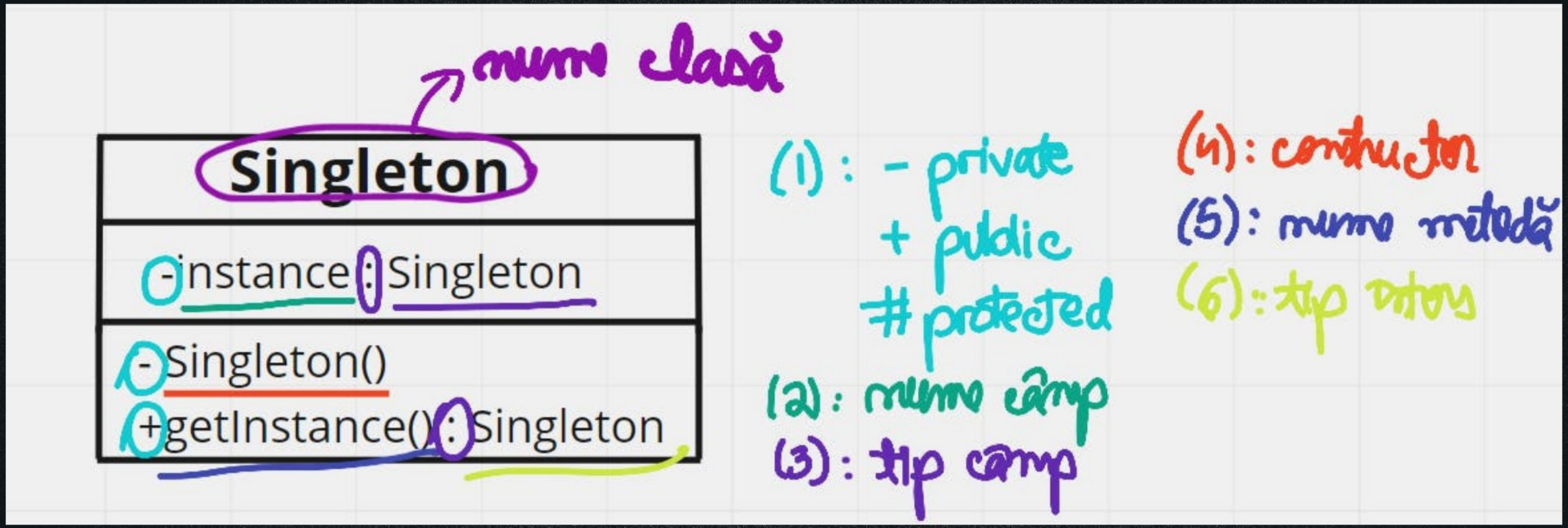
Mentimeter

Warning: singleton e considerat antipattern

- introduce multe dependențe între clase
- îngreunează testarea individuală a claselor

1

Mentimeter



Diagramă UML

3 ? 2

Mentimeter

```java
class SingletonLazy {
    private static SingletonLazy instance = null;

    private SingletonLazy() {}

    public static SingletonLazy getInstance() {
        if (instance == null) {
            instance = new SingletonLazy();
        }
        return instance;
    }
}

public class Main {
    public static void main(String[] args) {
        SingletonLazy singletonLazy = SingletonLazy.getInstance();
    }
}
```

Singleton lazy instantiation

2

Mentimeter

```java
class SingletonEager {
    private final static SingletonEager instance = new SingletonEager();

    private SingletonEager() {}

    public static SingletonEager getInstance() {
        return instance;
    }
}

public class Main {
    public static void main(String[] args) {
        SingletonEager singletonEager = SingletonEager.getInstance();
    }
}
```

Singleton eager instantiation

2

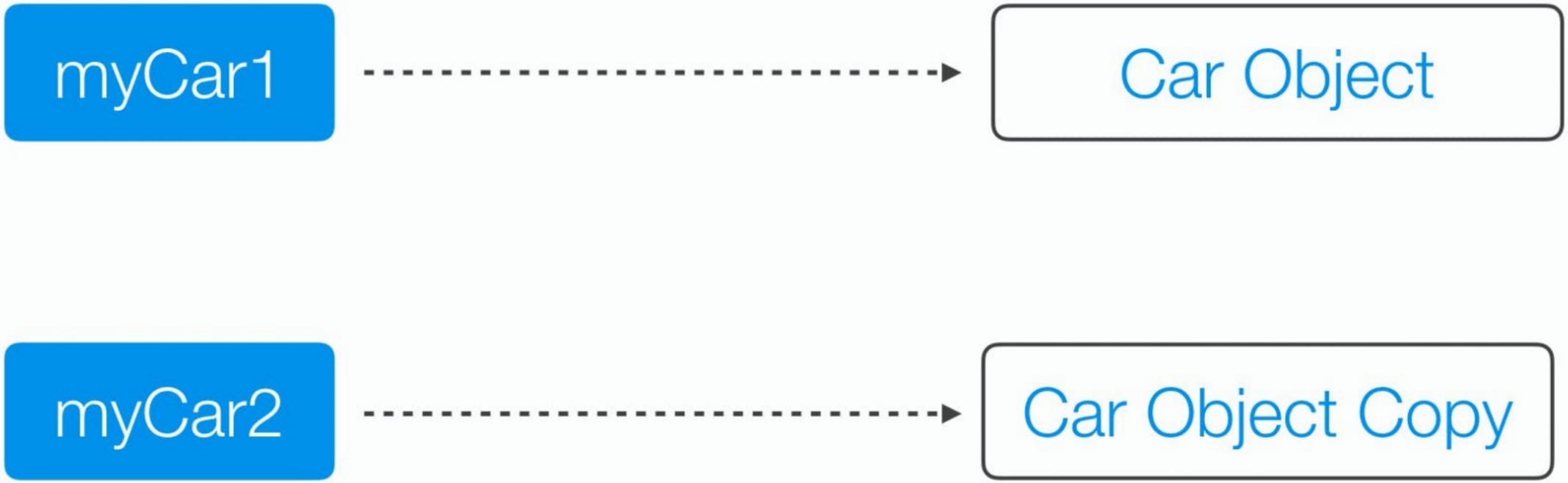
Mentimeter



Reference copy

3

Mentimeter



Object copy

2

Mentimeter

```
class Account {
    private String username;
    private String password;
}

class Person {
    private Account account;

    public Person(Person firstPerson) {
        this.account = firstPerson.account;
    }
}
```

Shallow copy

2

Mentimeter

```java
class Account {
    public String username;
    public String password;

    public Account(String username, String password) {
        this.username = username;
        this.password = password;
    }
}

class Person {
    private Account account;

    public Person(Person firstPerson) {
        this.account = new Account(firstPerson.account.username, firstPerson.account.password);
    }
}
```

Deep copy

Mentimeter

Demo

4

Leaderboard

Mentimeter

No results yet

Top Quiz participants will be displayed here once there are results!